

PYTHON FOR DATA SCIENCE • MODULE 1 • TOPIC 1.1

Variables, Data Types & Operators

Day 1 — where every Python program begins

90 min

Concepts + live code

30 min

Practice notebook

3

Colab notebooks

Prepared & presented by **Srinivasa Sai Chava**



Boston University

Where today sits in the 4-month journey



You are here

Module 1 is not optional background — it is the language every later module is written in.

Pandas is built on Python data types. A NumPy boolean mask is the comparison operator you learn today, applied to a million rows at once. A model's hyperparameters are floats you will cast, compare and tune. Nothing here is throwaway.

How today's two hours are structured

1

Variables

Names, binding, rules, conventions

15 min

2

Data types

int · float · str · bool · None

25 min

3

Type conversion

type(), casting, the input() trap

15 min

4

Operators

Arithmetic, comparison, logical, identity

30 min

5

Mini build

One program using all of it

5 min

6

Practice notebook

You code, I walk around

30 min

Three notebooks go with this session: *teaching notebook* · *practice questions* · *practice solutions*

By the end of today you will be able to

1

Declare & name

Create variables and name them the way professional Python code does

2

Identify types

Tell int, float, str, bool and None apart — and check with `type()`

3

Convert safely

Cast between types and avoid the three traps that break beginner code

4

Compute & compare

Use every operator family, and predict precedence without running the code

The one habit to build today

Read the error message. Every error in Python tells you the type, the line and usually the fix. Students who read errors learn twice as fast as students who re-run the cell hoping it changes.

1

Variables

A name is a label, not a box · 15 minutes

- Assignment & binding
- Dynamic typing
- Naming rules
- Conventions
- Swap & unpack

A variable is a name bound to an object

```
x = 10
```

Three things happen:

- Python creates the object 10 in memory
- Python creates the name x
- Python binds the name to the object

Read = as “gets”, never as “equals”.

“x gets 10.” Equality is `==`, which we meet in Section 4.

NAME

x

name

ok

OBJECT IN MEMORY

10 int

"Ravi" str

True bool

The type belongs to the object on the right — never to the name on the left.

Python is dynamically typed

You never declare a type. The value carries the type, so the same name can point at anything.

Other languages (Java / C)

```
int    x = 10;      // type locked
String x = "hi";    // ERROR

Type is declared once and fixed.
```

Python

```
x = 10           # x is an int
x = 3.14         # now a float
x = "hello"      # now a str
x = True         # now a bool  -- all legal
```

The upside

- Far less boilerplate to write
- Fast prototyping — ideal for analysis work

The cost — and it is real

- A typo can silently change a column's type
- Bugs surface at runtime, not before you run

Naming: the rules you must follow

Rule	Valid	Invalid	Error you get
Must start with a letter or _	<code>age</code> <code>_temp</code>	<code>2nd_value</code>	SyntaxError
Letters, digits and _ only	<code>total_marks_2</code>	<code>total-marks</code>	SyntaxError
No spaces	<code>student_name</code>	<code>student name</code>	SyntaxError
Cannot be a keyword	<code>class_name</code>	<code>class</code> <code>for</code> <code>if</code>	SyntaxError
Case sensitive	<i>Age and age are two different variables</i>	—	silent bug

There are 35 reserved keywords. Never memorise them — just check:

```
import keyword
print(keyword.kwlist)
```

Colab and every editor colour keywords differently. If your variable name changes colour as you type it, pick another name.

Conventions: the rules professionals follow anyway

snake_case

variables & functions

✓ student_score

✗ studentScore

UPPER_CASE

constants

✓ MAX_ROWS = 1000

✗ maxrows = 1000

Be descriptive

future you will thank you

✓ df_sales

✗ d

Never shadow a built-in

```
sum = 100          # looks harmless

sum([1, 2, 3])
TypeError: 'int' object
    is not callable
```

Avoid: list str sum type id max min dict input print

Multiple assignment & the one-line swap

```
a = b = c = 0
name, age, cgpa = "Ravi", 23, 8.4
```

```
p, q = 5, 9
p, q = q, p          # swapped, no temp var
```

2

Data Types

int · float · str · bool · None · 25 minutes

- Whole numbers
- Decimals & the float trap
- Text & f-strings
- Booleans & truthiness
- The None value

The four types everything else is built from

int

Whole numbers

42 -7 0

In data science: Counts, row indices, class labels

float

Decimal numbers

3.14 -0.5 2.0

In data science: Measurements, probabilities, weights

str

Text

"Mumbai" 'A'

In data science: Names, categories, column headers

bool

True / False

True False

In data science: Filters, masks, conditions

Plus one more: **None** — the NoneType value meaning “no value yet”. You will meet it constantly as a placeholder for missing data.

int and float in practice

int — unlimited size

```
students = 45
big = 2 ** 200      # no overflow!
population = 1_400_000_000
#           underscores are for humans
#           Python ignores them
```

float — has a decimal point

```
pi      = 3.14159
ratio = 2.0      # float, NOT int
sci     = 1.5e3   # 1.5 x 10^3 = 1500.0

type(2)  -> int    type(2.0) -> float
```

The floating-point trap — every data scientist hits this

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
```

Why: Binary cannot store 0.1 exactly, just as decimal cannot store 1/3 exactly. Not a Python bug — it is IEEE-754, and it happens in every language.

The fix: `import math` then `math.isclose(a, b)` — never compare floats with `==`

str — text, and the f-string you should always use

Strings are sequences

P	Y	T	H	O	N
0	1	2	3	4	5
-6	-5	-4	-3	-2	-1

forward index · negative index

```
name[0]  -> 'P'      name[-1] -> 'N'
name[0:3] -> 'PYT'   len(name) -> 6
```

Cleaning methods you will use daily

```
raw = "  Ravi Kumar  "
raw.strip()      # drop outer spaces
raw.upper()      # RAVI KUMAR
raw.replace("a","@")
raw.split()      # ['Ravi', 'Kumar']
"Ravi" in raw    # True
```

f-strings — use these everywhere

```
print(f"{name} scored {score:.2f}")
print(f"{score/100:.1%}")          # 92.5%
print(f"|{name:>10}|")              # right-pad to 10
print(f"{a=}, {b=}")               # a=7, b=3  <- debugging gold
```

⚠ str + number = TypeError

```
"10" + 5
-> TypeError

"10" + "5"  -> '105'
"ab" * 3    -> 'ababab'
```

bool and truthiness

bool is a subclass of int

```
True behaves as 1
False behaves as 0

True + True + False -> 2
```

Why that is genuinely useful

```
marks = [45, 78, 33, 90, 62]
passed = [m >= 40 for m in marks]
sum(passed) -> 4
# summing booleans = counting Trues
```

Every object can be used as a condition. These are the falsy values — everything else is truthy.

False

0

0.0

""

[]

{ }

()

None

The one that catches everybody: `bool("False")` is **True** — because it is a non-empty string. Python looks at whether the string has characters, not at what those characters say. Same for `"0"`.

3

Type Conversion

Checking types, casting, and three traps · 15 minutes

- `type()` vs `isinstance()`
- Implicit coercion
- Explicit casting
- The `input()` trap

Checking and converting types

Check the type

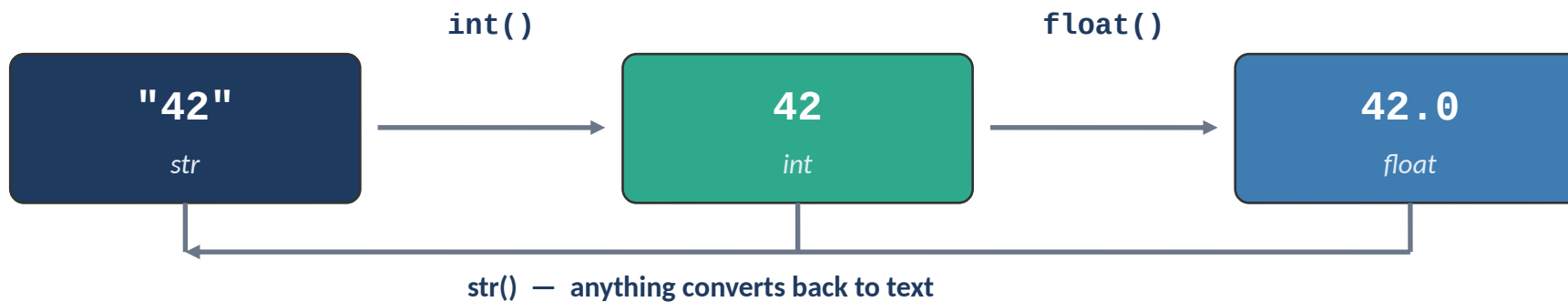
```
type(3.14)           # <class 'float'>
isinstance(3.14, float) # True  <- preferred
isinstance(x, (int, float))
# ^ 'is it numeric?' in one call
```

Implicit conversion — Python does it for you

```
5 + 2.0    -> 7.0    (int upgraded to float)
True + 4.5 -> 5.5    (bool -> int -> float)
```

Python widens to avoid losing precision.

Explicit conversion (casting) — you do it yourself



`bool()` takes anything:

`bool("data")` → **True**

`bool("")` → **False**

Three casting traps — and the input() trap

1 int() truncates, it does not round

```
int(9.99)    -> 9
int(-9.99)   -> -9
round(9.99)  -> 10
```

Use round() when you mean rounding.

2 int() cannot parse a decimal string

```
int("3.5")
  -> ValueError
int(float("3.5")) -> 3
```

Go through float() first.

3 Garbage strings raise ValueError, not NaN

```
int("abc")    -> ValueError
int("1,00,000") -> ValueError
int(" 100 ")  -> 100 (ok)
```

Wrap risky casts in try / except.

⚠ input() ALWAYS returns a string — the number-one Day-1 bug

```
age = input("Age: ")    # you type 25
age + 1
  -> TypeError: can only concatenate str
```

```
age = int(input("Age: ")) # cast it
print(age + 1)           # 26
# for decimals use float(input(...))
```

4

Operators

Arithmetic · comparison · logical · assignment · membership · identity · 30 minutes

- 7 arithmetic ops
- // and %
- Precedence
- Chained comparison
- and / or / not
- is vs ==

Arithmetic operators

Op	Name	Expression	Result	Watch out for
+	Addition	a + b	22	On strings it concatenates
-	Subtraction	a - b	12	—
*	Multiplication	a * b	85	On a string it repeats
/	True division	a / b	3.4	ALWAYS returns a float
//	Floor division	a // b	3	Rounds DOWN, not toward zero
%	Modulus	a % b	2	Sign follows the divisor
**	Exponent	a ** 2	289	Right-associative

10 / 5 gives **2.0** (a float) • **10 // 5** gives **2** (an int). If you index a list with the first one, you get a TypeError.

// and % split a number into groups and leftovers

197 minutes → how many whole hours, and what is left?

60 min

60 min

60 min

17

`197 // 60 = 3` whole hours

`197 % 60 = 17` minutes left over

```
h = 197 // 60
m = 197 % 60
f"{h} hr {m} min"
# '3 hr 17 min'
```

The classic use: divisibility

```
n % 2 == 0    -> n is even
n % 2 == 1    -> n is odd
n % 5 == 0    -> divisible by 5
```

You will use this in every loop.

⚠ Negatives behave differently

```
7 // 2  -> 3
-7 // 2  -> -4    (not -3!)
7 % 3    -> 1
-7 % 3    -> 2    (sign follows divisor)
# // floors toward -infinity
```

Operator precedence — who goes first?



Predict these

```
2 + 3 * 4
-> 14, not 20

2 ** 3 ** 2
-> 512, not 64
(= 2 ** (3 ** 2))

10 - 4 - 3
-> 3 (left to right)
```

The rule that saves you

When in doubt, add parentheses. Nobody has ever been criticised in code review for making an expression too obvious.

Comparison operators — these return a bool

Operator	Meaning	x=10, y=3	Result
<code>==</code>	equal to	<code>x == y</code>	False
<code>!=</code>	not equal to	<code>x != y</code>	True
<code>></code>	greater than	<code>x > y</code>	True
<code><</code>	less than	<code>x < y</code>	False
<code>>=</code>	greater or equal	<code>x >= 10</code>	True
<code><=</code>	less or equal	<code>x <= y</code>	False

Strings compare too — alphabetically, by Unicode order

```
"apple" < "banana"    -> True
"Apple" < "apple"     -> True    (caps sort first)
```

 **= vs == — the classic**

```
score = 90      # ASSIGN
score == 90     # COMPARE -> True
if score = 90:  -> SyntaxError
```

Chained comparison — a Python superpower

Most languages force you to write the condition twice. Python lets you write it the way maths does.

```
# other languages
x > 0 && x < 100

# Python
0 <= marks <= 100
60 <= marks < 80    # grade band
```

Logical operators and short-circuiting

Truth table

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Real use: placement eligibility

```
marks      = 82
attend     = 76
backlogs   = 0
is_placed  = False

eligible = (marks >= 70) and \
           (attend >= 75) and \
           (backlogs == 0) and \
           (not is_placed)
eligible  -> True    # all four hold
```

Short-circuit evaluation — and why it prevents crashes

and stops at the first False. or stops at the first True. The rest is never evaluated.

```
if n != 0 and total / n > 10:
#     ^ when n is 0 the division
#     NEVER RUNS -> no crash
```

Assignment, membership and identity operators

Compound assignment

```
x += 5    x = x + 5
x -= 5    x = x - 5
x *= 5    x = x * 5
x /= 5    x = x / 5
x //= 5   x = x // 5
x %= 5    x = x % 5
x **= 5   x = x ** 5
```

Works on str and list too:

```
msg += " Science"    items += [3, 4]
```

Membership: in / not in

“Does this container hold this item?” Returns a bool.

```
subjects = ["Python",
            "Statistics"]

"Python" in subjects -> True
"Java" not in subjects -> True

"@ " in email        -> True
```

Identity: is / is not

== same VALUE?

is same OBJECT in memory?

```
a = [1,2,3]
b = [1,2,3]    # separate list
c = a          # same object

a == b -> True    a is b -> False
a is c -> True
```

Rule: use `is` only with `None`, `True` and `False`. Use `==` for everything else.

Python caches small integers (-5 to 256), so 256 is 256 gives True but 257 is 257 often gives False. Same value, different objects. That is an implementation detail — never build logic on it.

Debug corner — the eight errors you will actually hit

You wrote	What happens	Fix
<code>input() + 1</code>	<code>TypeError</code>	<code>int(input())</code>
<code>int("3.5")</code>	<code>ValueError</code>	<code>int(float("3.5"))</code>
<code>if x = 5:</code>	<code>SyntaxError</code>	<code>if x == 5:</code>
<code>0.1+0.2 == 0.3</code>	silently False	<code>math.isclose(a, b)</code>
<code>2 ** 3 ** 2</code>	512, not 64	<code>(2 ** 3) ** 2</code>
<code>list = [1,2]</code>	built-in destroyed	<code>numbers = [1,2]</code>
<code>int(9.99)</code>	9 — truncated	<code>round(9.99)</code>

Every one of these is in the practice notebook. If you can predict all eight without running the code, you have today's material.

Practice session — 30 minutes



Predict the output

Write your answer BEFORE running the cell

8 questions



Spot & fix the bug

Name the error type, then fix it

4 questions



Write the code

Seconds converter, BMI, eligibility checker

5 tasks

Your notebook

[Day1_Practice_Questions.ipynb](#)

- Open it in Colab
- Attempt alone first
- Raise a hand — I am walking around
- Solutions shared after class

Do not look up answers for the first two minutes of being stuck. That two minutes is where the learning happens.

TODAY IN TWELVE LINES

Recap

- 1 = assigns. == compares.
- 2 A variable is a name bound to an object.
- 3 snake_case, no keywords, never shadow built-ins.
- 4 Core types: int, float, str, bool — plus None.
- 5 $0.1 + 0.2 \neq 0.3$. Use `math.isclose()`.
- 6 `input()` always returns a str. Cast it.
- 7 `int(9.99)` is 9. `round(9.99)` is 10.
- 8 `int("3.5")` is a `ValueError` — go via `float()`.
- 9 `/` gives float, `//` floors, `%` gives remainder.
- 10 `**` is right-to-left: `2**3**2 == 512`.
- 11 Falsy: 0, 0.0, "", [], {}, (), None, False.
- 12 `is` compares identity, `==` compares value.

Homework

Rebuild the eligibility checker using `input()` with `try/except` · Write a tip calculator (bill, tip %, people → per-person amount)

Next class — Topic 1.2

Loops & Conditionals: `if` / `elif` / `else`, `for` and `while` loops, `break`, `continue`, `pass`